

Storing the Index

Introduction to Information Retrieval

Dwaipayan Roy

Department of Computational and Data Sciences



Memory in a Computer

- **Primary memory (RAM):** very fast

Memory in a Computer

- **Primary memory (RAM):** very fast, limited capacity, volatile, expensive.

Memory in a Computer

- **Primary memory (RAM):** very fast, limited capacity, volatile, expensive.
- **Secondary memory (HDD, SSD):** slow, almost unlimited size, non-volatile, cheap.

Memory in a Computer

- **Primary memory (RAM):** very fast, limited capacity, volatile, expensive.
- **Secondary memory (HDD, SSD):** slow, almost unlimited size, non-volatile, cheap.
- All variables/data structures that we declare/use in a program → stored in primary memory.
 - ▶ Volatile: gone with system power off.

Index Construction

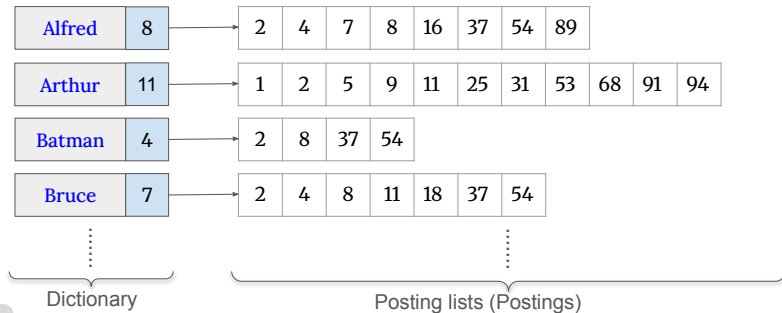
- **In-memory index construction:**

- ▶ Entire inverted index constructed in the primary memory.
- ▶ Very fast.
- ▶ Only suitable for small collections.

- **Disk-based index construction:**

- ▶ Inverted index constructed in the secondary memory.
- ▶ Slow.
- ▶ Can accommodate collections of any size.

Creating an Inverted Index



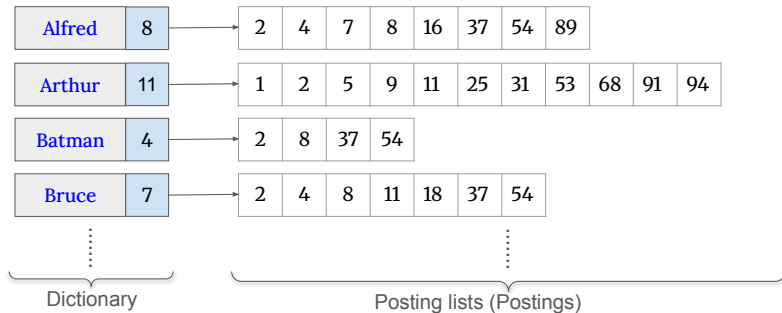
indexing time

Life cycle of an inverted index consists of two phases:

- 1 **Index construction:** postings list built for each term in the collection in an incremental fashion.
- 2 **Query processing:** information stored in phase 1 used to process search queries.

searching time

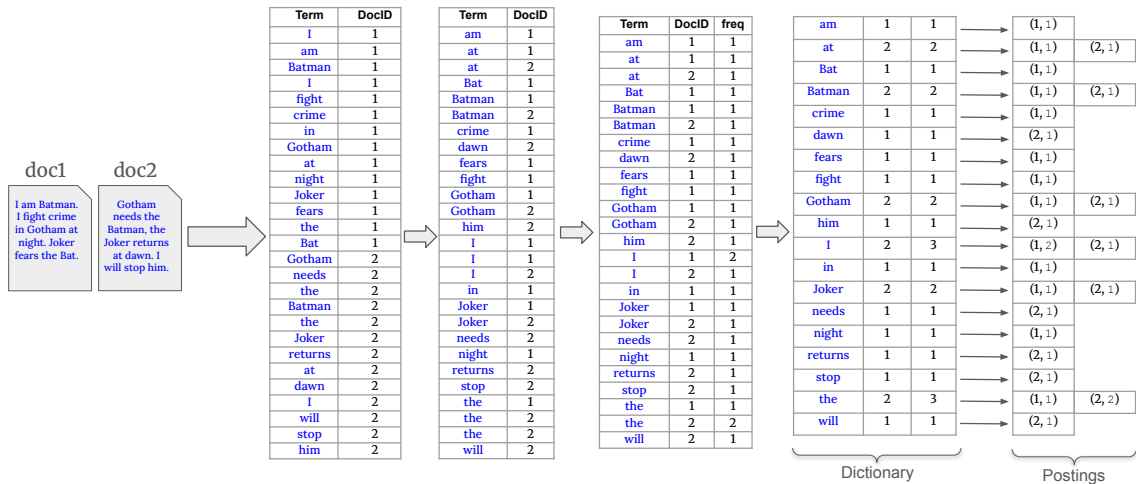
Creating an Inverted Index



Things to be stored:

- 1 **Dictionary:** containing all the terms of the collection (less frequently updated)
- 2 **Posting list:** for each term, containing the docids in which it appears (frequently updated)

Creating an Inverted Index

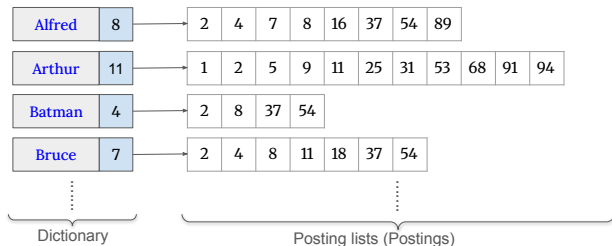


Creating an Inverted Index

1. storing dictionary

Goal

To allow quick searching for a term in the dictionary.

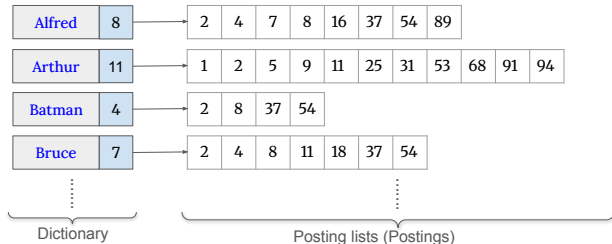


Creating an Inverted Index

1. storing dictionary

Goal

To allow quick searching for a term in the dictionary.



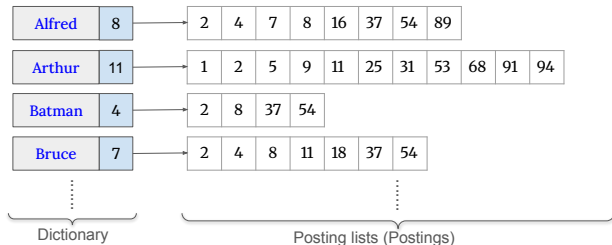
- Dictionary will contain:
 - ▶ the actual term;
 - ▶ document frequency;
 - ▶ some information regarding the start of the posting list of the term.
 - ▶ ...

Creating an Inverted Index

1. storing dictionary

Goal

To allow quick searching for a term in the dictionary.



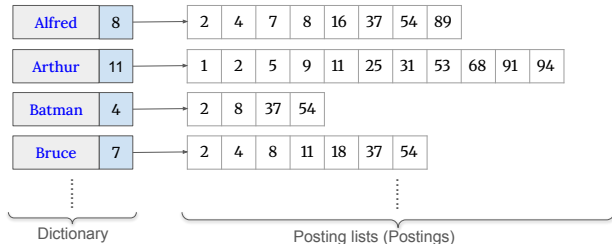
- Dictionary will contain:
 - ▶ the actual term;
 - ▶ document frequency;
 - ▶ some information regarding the start of the posting list of the term.
 - ▶ ...
- Two ways to store a dictionary using:
 - 1 Sort-based dictionary.
 - 2 Hash-based dictionary.

Creating an Inverted Index

1. storing dictionary

Goal

To allow quick searching for a term in the dictionary.



- Dictionary will contain:
 - ▶ the actual term;
 - ▶ document frequency;
 - ▶ some information regarding the start of the posting list of the term.
 - ▶ ...
- Two ways to store a dictionary using:
 - 1 Sort-based dictionary.
 - 2 Hash-based dictionary.

The choice of solution depends on:

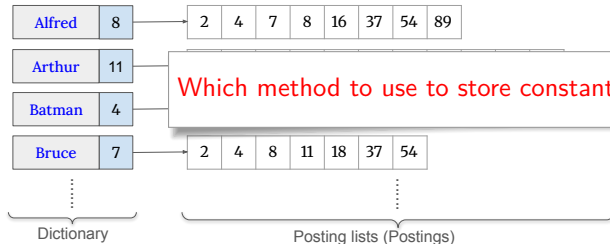
- Number of terms to store?
- Static/dynamic?

Creating an Inverted Index

1. storing dictionary

Goal

To allow quick searching for a term in the dictionary.



Which method to use to store constantly growing set of Twitter posts?

- Dictionary will contain:
 - ▶ the actual term;
 - ▶ document frequency;
 - ▶ some information regarding the start of the posting list of the term.

ary using:

- 1 Sort-based dictionary.
- 2 Hash-based dictionary.

The choice of solution depends on:

- Number of terms to store?
- Static/dynamic?

Creating an Inverted Index

1. storing dictionary: comparing sort-based with hash-based

- Hash-based dictionaries usually faster than sort-based ones.
- No costly binary search or tree traversal required to find a term.
- Speed advantage depends on size of hash table.
- Small tables $\rightarrow$ many collisions $\rightarrow$ reduce performance.

Creating an Inverted Index

1. storing dictionary: comparing sort-based with hash-based

- Hash-based dictionaries usually faster than sort-based ones.
- No costly binary search or tree traversal required to find a term.
- Speed advantage depends on size of hash table.
- Small tables $\rightarrow$ many collisions $\rightarrow$ reduce performance.
- ★ Table size should grow linearly with the number of terms.

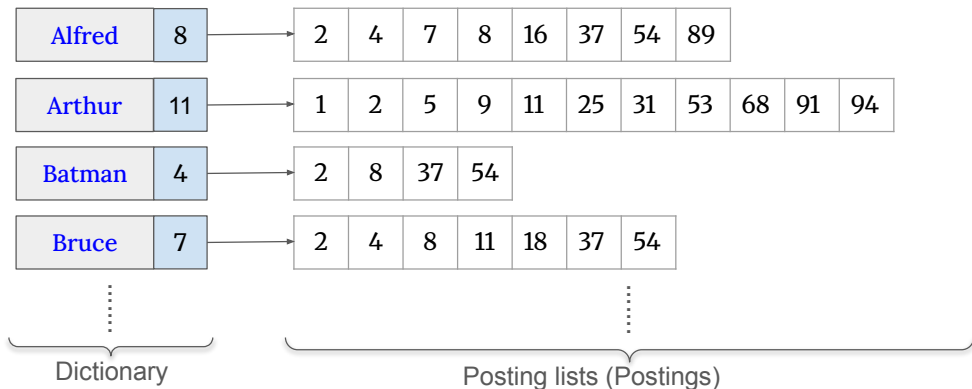
Creating an Inverted Index

1. storing dictionary: comparing sort-based with hash-based

Aspect	Sort-based Dictionary	Hash-based Dictionary
Term storing	Sorted order	Using a hash table
Lookup Time	$O(\log n)$ (binary search)	$O(1)$ avg. (hash function)
Supports Range Queries?	✓ e.g., for wildcard/prefix searches	✗ no inherent order
Use Case	When sorting and term range access are needed	When fast exact-term lookup is needed

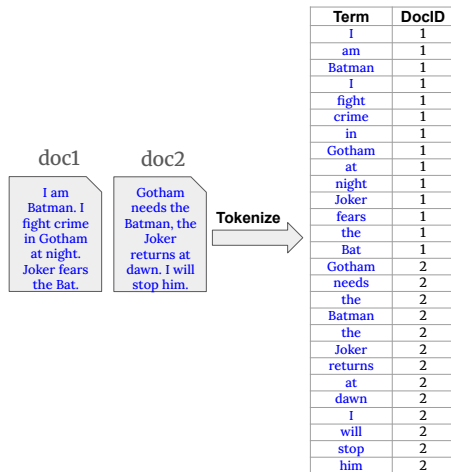
Creating an Inverted Index

2. storing posting list



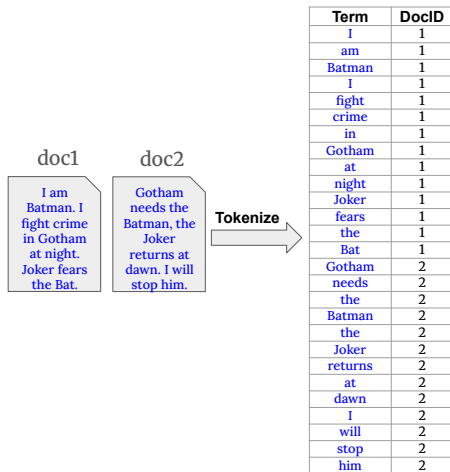
Creating an Inverted Index

2. storing posting list - intermediate memory requirement



Creating an Inverted Index

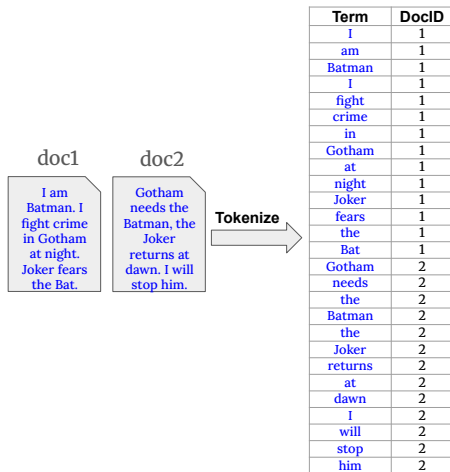
2. storing posting list - intermediate memory requirement



- Final postings of any term incomplete till end.

Creating an Inverted Index

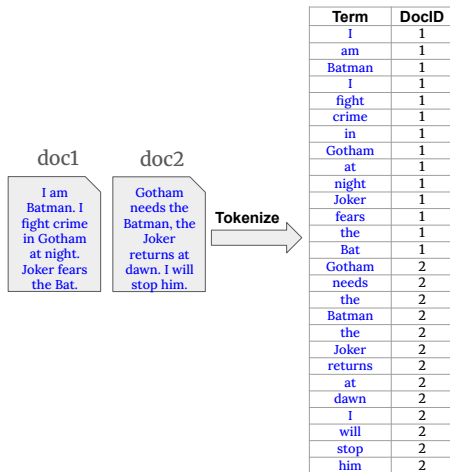
2. storing posting list - intermediate memory requirement



- Final postings of any term incomplete till end.
- `sizeof(char)`: 1 byte - 4 bytes

Creating an Inverted Index

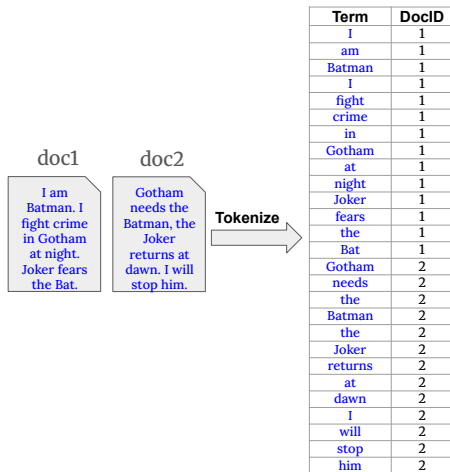
2. storing posting list - intermediate memory requirement



- Final postings of any term incomplete till end.
- `sizeof(char)`: 1 byte - 4 bytes
 - ▶ `sizeof(term)`: 20 bytes - 80 bytes
(considering 20 being highest term-length).

Creating an Inverted Index

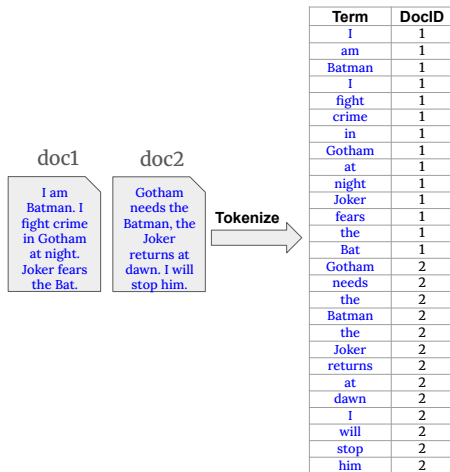
2. storing posting list - intermediate memory requirement



- Final postings of any term incomplete till end.
- `sizeof(char)`: 1 byte - 4 bytes
 - ▶ `sizeof(term)`: 20 bytes - 80 bytes
(considering 20 being highest term-length).
- `sizeof(int)`: 2 bytes / 4 bytes

Creating an Inverted Index

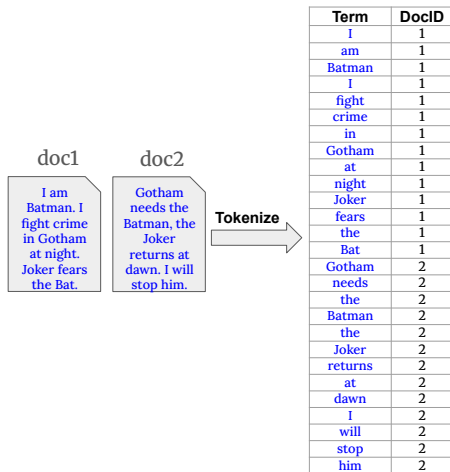
2. storing posting list - intermediate memory requirement



- Final postings of any term incomplete till end.
- `sizeof(char)`: 1 byte - 4 bytes
 - ▶ `sizeof(term)`: 20 bytes - 80 bytes
(considering 20 being highest term-length).
- `sizeof(int)`: 2 bytes / 4 bytes
 - ▶ `sizeof(docid)`: 2 bytes / 4 bytes

Creating an Inverted Index

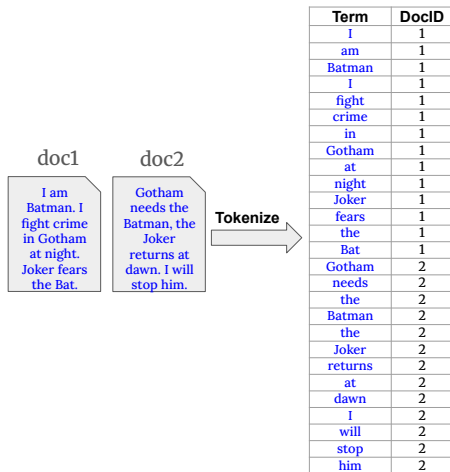
2. storing posting list - intermediate memory requirement



- Final postings of any term incomplete till end.
- `sizeof(char)`: 1 byte - 4 bytes
 - ▶ `sizeof(term)`: 20 bytes - 80 bytes
(considering 20 being highest term-length).
- `sizeof(int)`: 2 bytes / 4 bytes
 - ▶ `sizeof(docid)`: 2 bytes / 4 bytes
- Space needed per token:
 - ▶ term: 20 bytes - 80 bytes
 - ▶ docid: 2 bytes - 4 bytes
 - ▶ frequency: 2 bytes - 4 bytes

Creating an Inverted Index

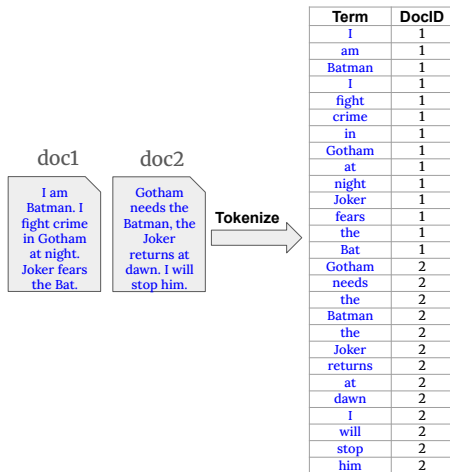
2. storing posting list - intermediate memory requirement



- Final postings of any term incomplete till end.
- `sizeof(char)`: 1 byte - 4 bytes
 - ▶ `sizeof(term)`: 20 bytes - 80 bytes (considering 20 being highest term-length).
- `sizeof(int)`: 2 bytes / 4 bytes
 - ▶ `sizeof(docid)`: 2 bytes / 4 bytes
- Space needed per token:
 - ▶ term: 20 bytes - 80 bytes
 - ▶ docid: 2 bytes - 4 bytes
 - ▶ frequency: 2 bytes - 4 bytes
- A relatively small collection (e.g. RCV1) contains 100M tokens.

Creating an Inverted Index

2. storing posting list - intermediate memory requirement



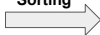
- Final postings of any term incomplete till end.
- `sizeof(char)`: 1 byte - 4 bytes
 - ▶ `sizeof(term)`: 20 bytes - 80 bytes (considering 20 being highest term-length).
- `sizeof(int)`: 2 bytes / 4 bytes
 - ▶ `sizeof(docid)`: 2 bytes / 4 bytes
- Space needed per token:
 - ▶ term: 20 bytes - 80 bytes
 - ▶ docid: 2 bytes - 4 bytes
 - ▶ frequency: 2 bytes - 4 bytes
- A relatively small collection (e.g. RCV1) contains 100M tokens.
- Alternative: use `termid` in place of `term`.

Creating an Inverted Index

2. storing posting list - intermediate memory requirement

Term	DocID		Term	DocID
I	1		am	1
am	1		at	1
Batman	1		at	2
I	1		Bat	1
fight	1		Batman	1
crime	1		Batman	2
in	1		crime	1
Gotham	1		dawn	2
at	1		fears	1
night	1		fight	1
Joker	1		Gotham	1
fears	1		Gotham	2
the	1		him	2
Bat	1		I	1
Gotham	2		I	1
needs	2		I	2
the	2		in	1
Batman	2		Joker	1
The	2		Joker	2
Joker	2		needs	2
returns	2		night	1
at	2		returns	2
dawn	2		stop	2
I	2		the	1
will	2		the	2
stop	2		the	2
him	2		will	2

Sorting

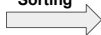


Creating an Inverted Index

2. storing posting list - intermediate memory requirement

Term	DocID
I	1
am	1
Batman	1
I	1
fight	1
crime	1
in	1
Gotham	1
at	1
night	1
Joker	1
fears	1
the	1
Bat	1
Gotham	2
needs	2
the	2
Batman	2
The	2
Joker	2
returns	2
at	2
dawn	2
I	2
will	2
stop	2
him	2

Sorting



Term	DocID
am	1
at	1
at	2
Bat	1
Batman	1
Batman	2
crime	1
dawn	2
fears	1
fight	1
Gotham	1
Gotham	2
him	2
I	1
I	1
I	2
in	1
Joker	1
Joker	2
needs	2
night	1
returns	2
stop	2
the	1
the	2
the	2
will	2

- While indexing, documents are parsed one at a time.
 - ▶ broken up as term-docid pairs.
- Final postings of any term incomplete until the end.

Creating an Inverted Index

2. storing posting list - intermediate memory requirement

Term	DocID
I	1
am	1
Batman	1
I	1
fight	1
crime	1
in	1
Gotham	1
at	1
night	1
Joker	1
fears	1
the	1
Bat	1
Gotham	2
needs	2
the	2
Batman	2
The	2
Joker	2
returns	2
at	2
dawn	2
I	2
will	2
stop	2
him	2

Sorting



Term	DocID
am	1
at	1
at	2
Bat	1
Batman	1
Batman	2
crime	1
dawn	2
fears	1
fight	1
Gotham	1
Gotham	2
him	2
I	1
I	1
I	2
in	1
Joker	1
Joker	2
needs	2
night	1
returns	2
stop	2
the	1
the	2
the	2
will	2

- While indexing, documents are parsed one at a time.
 - ▶ broken up as term-docid pairs.
- Final postings of any term incomplete until the end.
- Can we keep all intermediate-pairs in memory and then do the sort in-memory at the end?

Creating an Inverted Index

2. storing posting list - intermediate memory requirement

Term	DocID
I	1
am	1
Batman	1
I	1
fight	1
crime	1
in	1
Gotham	1
at	1
night	1
Joker	1
fears	1
the	1
Bat	1
Gotham	2
needs	2
the	2
Batman	2
The	2
Joker	2
returns	2
at	2
dawn	2
I	2
will	2
stop	2
him	2

Sorting



Term	DocID
am	1
at	1
at	2
Bat	1
Batman	1
Batman	2
crime	1
dawn	2
fears	1
fight	1
Gotham	1
Gotham	2
him	2
I	1
I	1
I	2
in	1
Joker	1
Joker	2
needs	2
night	1
returns	2
stop	2
the	1
the	2
the	2
will	2

- While indexing, documents are parsed one at a time.
 - ▶ broken up as term-docid pairs.
- Final postings of any term incomplete until the end.
- Can we keep all intermediate-pairs in memory and then do the sort in-memory at the end?
- **Not for large collections.**
- 28-88 bytes per entry, we need a lot of space for large collections.

A Common Issue

Exhaustion of primary memory!

A Common Issue

Exhaustion of primary memory!

Need external sorting algorithms!

External Sorting

- Blocked sort-based indexing
- Single-pass in-memory indexing

Blocked Sort-Based Indexing (BSBI)

Algorithm 1 BSBI: Blocked Sort-Based Indexing

Divide the document collection into blocks

for each block **do**

 Tokenize the documents in the block

 Generate (*term*, *docID*) pairs

 Sort the pairs by term, then docID

 Write the sorted list to disk as an intermediate index

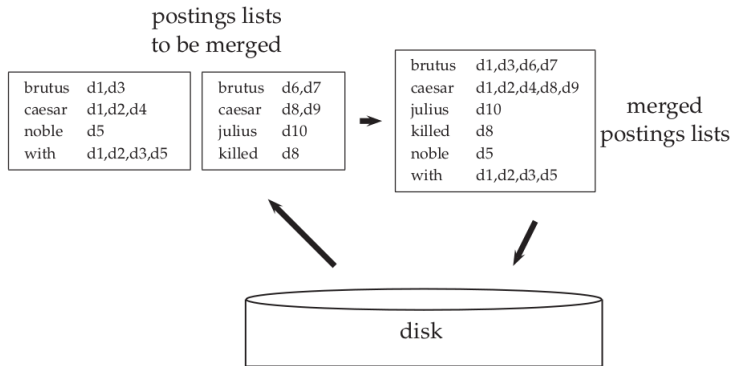
end for

Merge all intermediate sorted blocks into a single inverted index

- segments the collection into parts of equal size,
- sorts the termID-docID pairs of each part in memory,
- stores intermediate sorted results on disk,
- merges all intermediate results into final index.

MRS - Chapter 4.2

Blocked Sort-Based Indexing (BSBI)



MRS - Chapter 4.2

Blocked Sort-Based Indexing (BSBI)

problem

- Our assumption: we can keep the dictionary in memory.
- We need the dictionary (which grows dynamically) in order to implement a term to termID mapping.
- Actually, we could work with term, docID postings instead of termID, docID postings but then intermediate files become very large. (We would end up with a scalable, but very slow index construction method.)

Single-Pass In-Memory Indexing (SPIMI)

Algorithm 1 SPIMI: Single-Pass In-Memory Indexing

```
1: Initialize an empty in-memory dictionary
2: for each document in the collection do
3:   for each token in the document do
4:     if token not in dictionary then
5:       Add token to dictionary with a new postings list
6:     end if
7:     Add document ID to the token's postings list (if not already present)
8:   end for
9:   if memory limit exceeded then
10:    Write current dictionary (index block) to disk
11:    Clear the dictionary
12:   end if
13: end for
14: Write any remaining dictionary to disk
15: Optionally merge index blocks into final inverted index
```

MRS - Chapter 4.3

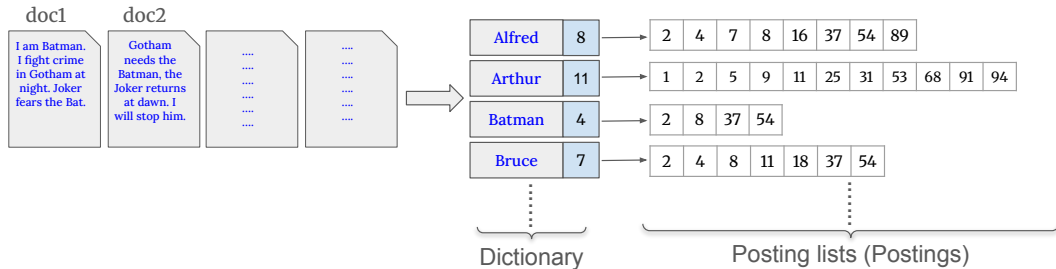
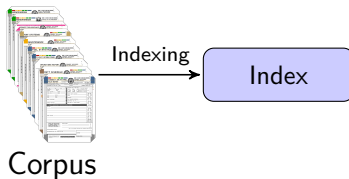
Comparing BSBI and SPIMI

Feature	BSBI	SPIMI
Indexing Type	Multi-pass, sort-based	Single-pass, in-memory
Dictionary Type	Sort-based (e.g., tree/array)	Hash-based
Sorting Required?	Yes (term/docID pairs)	No
Speed	Slower (due to sorting)	Faster (single pass)
Memory Usage	Efficient for disk-based systems	Efficient if block fits in memory

Table: Comparison of BSBI and SPIMI Indexing Algorithms

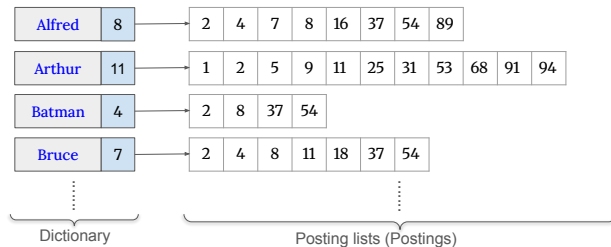
Information Retrieval

indexing



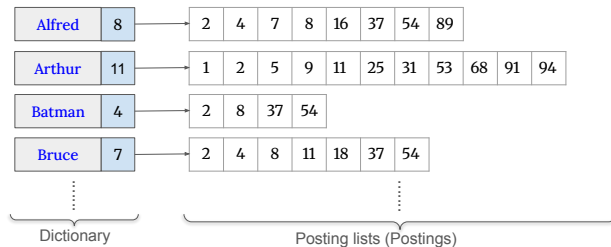
Information Retrieval

retrieval



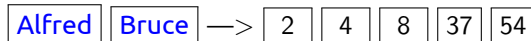
Information Retrieval

retrieval



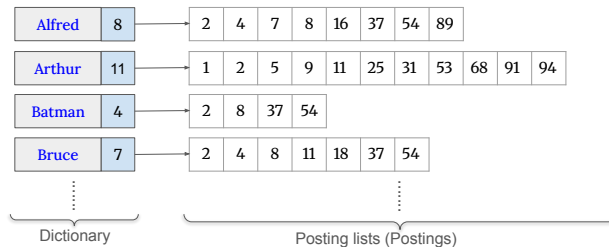
Query

- Alfred Bruce



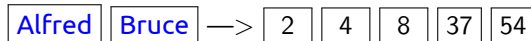
Information Retrieval

retrieval



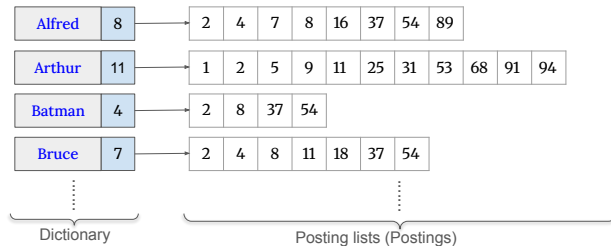
Query

- Alfred Bruce



still boolean retrieval!

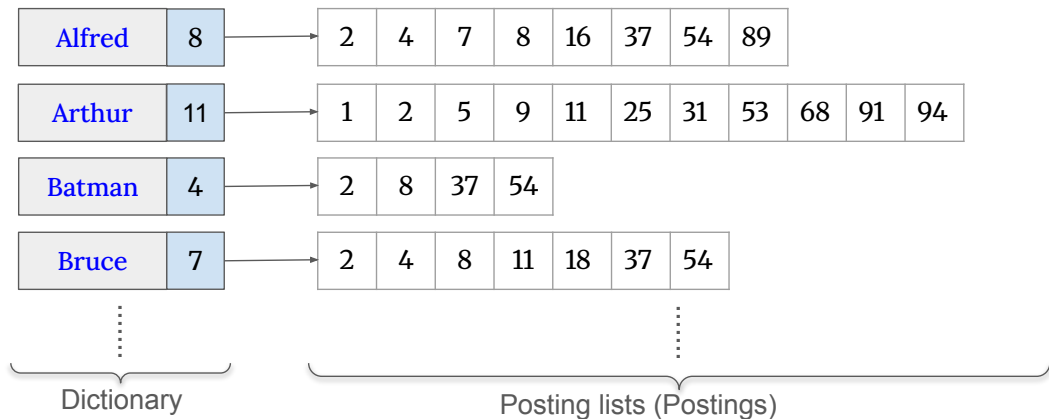
Why still stuck with Boolean retrieval?



- Only recording whether a term is present in the document or not.
- **No notion of *term importance*** - presence of all terms considered **equally important**.

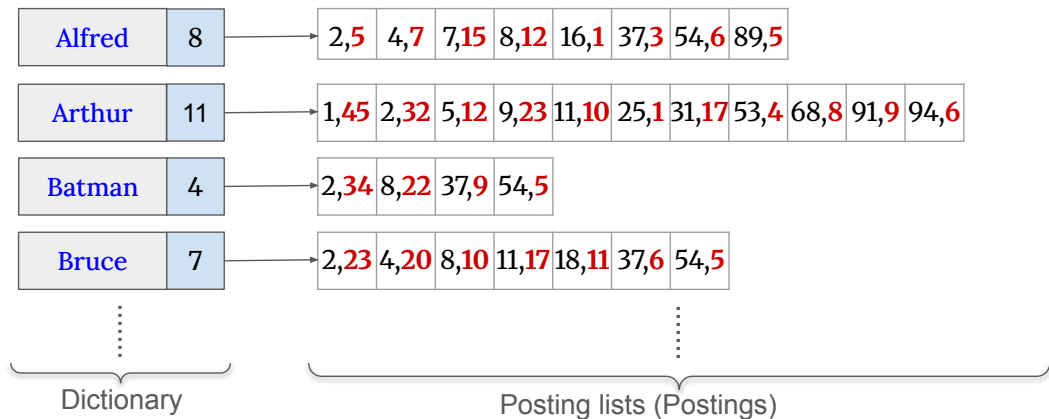
Information Retrieval

indexing for boolean retrieval



Information Retrieval

indexing beyond boolean retrieval



Information Retrieval

indexing beyond boolean retrieval - positional indexing

- In the posting list of a term, store:
 - ❶ the term;
 - ❷ the term-frequency;

Information Retrieval

indexing beyond boolean retrieval - positional indexing

- In the posting list of a term, store:
 - ❶ the term;
 - ❷ the term-frequency;
 - ❸ the position of the term in the document.

Information Retrieval

indexing beyond boolean retrieval - positional indexing

- In the posting list of a term, store:
 - ❶ the term;
 - ❷ the term-frequency;
 - ❸ the position of the term in the document.
- Useful for phrase query and proximity query.

Terminologies (so far)

- Corpus/Collection (C) - a set of documents.
- Document (D) - a textual indexing unit.
- Tokens/Terms/Words (w) - building block of a document.
- Document frequency ($df(w)$) - number of documents containing w .
- Term frequency ($tf(w,D)$) - count of w in document D .
- Collection frequency ($cf(w)$) - count of w in the entire C .
- Query (Q) - $Q = \{q_1, q_2, \dots, q_k\}$

